

SLIIT
SE3082 – Parallel Computing



REPORT
LAB 05

Akash D
IT24101853

Exercise 01

Code Implementation

```
M /a/3 Semester/PC/Lab05/Exercise01
GNU nano 9.1
#include <omp.h>
#include <stdio.h>

int main() {

#pragma omp parallel
{
    int th = omp_get_thread_num();
    printf("thread # %d\n", th);

    #pragma omp barrier

    printf("Outside Barrier # %d\n", omp_get_thread_num());
    printf("Hi again from thread # %d\n", omp_get_thread_num());

}

    return 0;
}
```

Output

```
M /a/3 Semester/PC/Lab05/Exercise01
LENOVO@Akash MINGW64 ~
$ cd "/a/3 Semester/PC/Lab05/"
LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05
$ cd Exercise01
LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise01
$ nano lab05a.c
LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise01
$ gcc -fopenmp lab05a.c -o lab05a.o
LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise01
$ ./lab05a.o
thread # 3
thread # 1
thread # 2
thread # 6
thread # 4
thread # 5
thread # 14
thread # 13
thread # 8
thread # 12
thread # 11
thread # 9
thread # 7
thread # 0
thread # 10
thread # 15
Outside Barrier # 3
Hi again from thread # 3
Outside Barrier # 2
Hi again from thread # 2
Outside Barrier # 6
Hi again from thread # 6
Outside Barrier # 13
Hi again from thread # 13
Outside Barrier # 8
Hi again from thread # 8
Outside Barrier # 14
Hi again from thread # 14
Outside Barrier # 10
Hi again from thread # 10
Outside Barrier # 7
Hi again from thread # 7
Outside Barrier # 1
Hi again from thread # 1
Outside Barrier # 5
Hi again from thread # 5
Outside Barrier # 11
Hi again from thread # 11
Outside Barrier # 9
Hi again from thread # 9
Outside Barrier # 4
Hi again from thread # 4
Outside Barrier # 0
Hi again from thread # 0
Outside Barrier # 12
Hi again from thread # 12
Outside Barrier # 15
Hi again from thread # 15
LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise01
$ |
```

In OpenMP `#pragma omp barrier` acts as an explicit synchronization point for all threads within a parallel region. When execution reaches the barrier directive every individual thread must pause and wait until every other thread in the team has also reached that exact point.

In this specific program:

1. Initial Phase: Upon entering the parallel region multiple threads are spawned concurrently. Each thread retrieves its thread number and executes the first print statement. Because thread scheduling is dynamic these initial outputs may appear out of order.
2. Synchronization Point: As each thread encounters the barrier directive it is halted. No thread is permitted to proceed past this point until all active threads have completed the preceding instructions and arrived at the barrier.
3. Post-Verification: Once synchronization is complete across all threads they are released to execute the remaining print statements. This guarantees a clean synchronization boundary ensuring that no thread begins the next phase of execution prematurely.